

DOCUMENTO TÉCNICO DE DISEÑO

DISEÑO CONCEPTUAL Y LÓGICO DE LA BASE DE DATOS DEL PROYECTO ECOMMIFY

DISEÑO Y OPTIMIZACIÓN DE BASES DE DATOS

NESTOR ALEJANDRO RODRIGUEZ BENAVIDES - NESTORROBE@UNISABANA.EDU.CO

CARLOS DANIEL SANDOVAL - CARLOSSANDPAR@UNISABANA.EDU.CO

PETER ALEXANDER PALACIOS GARNICA - PETERPAGA@UNISABANA.EDU.CO

JUAN GUILLERMO OSSA SÁNCHEZ - JUANOSSA@UNISABANA.EDU.CO

MAESTRÍA EN ARQUITECTURA DE SOFTWARE

DOCENTE: MIGUEL ALFONSO VARELA

23 DE MAYO DE 2026

1. Resumen ejecutivo.....	2
1.1 Alcance del documento.....	2
1.2 Enfoque arquitectónico seleccionado.....	2
2. Análisis de requisitos.....	3
2.1 Contexto de Ecommify.....	3
2.2 Requisitos funcionales.....	3
2.3 Requisitos no funcionales.....	4
2.4 Identificación de entidades del dominio.....	5
2.5 Tabla de volúmenes de datos esperados.....	5
3. Diseño conceptual.....	6
3.1 Descripción de entidades y relaciones.....	6
3.2 Restricciones de negocio identificadas.....	7
4. Diseño lógico - Módulo PostgreSQL.....	7
4.1 Esquema relacional propuesto.....	7
4.2 Aplicación de tipos avanzados PostgreSQL.....	10
4.3 Extensiones PostgreSQL propuestas.....	10
4.4 Estrategia de particionamiento.....	11
5. Diseño lógico preliminar - Módulo MongoDB.....	11
5.1 Ejemplo de documento: product_catalog.....	11
5.2 Ejemplo de documento: product_reviews.....	12
5.3 Patrones de modelado MongoDB propuestos.....	12
6. Decisiones arquitectónicas justificadas.....	12
6.1 Matriz de decisión por entidad.....	13
6.2 Análisis CAP aplicado.....	13
6.3 Trade-offs y estrategias de mitigación.....	13
6.4 Flujo general de sincronización.....	14
7. Anexos.....	14
7.1 Diccionario de datos resumido.....	14
7.2 Scripts SQL preliminares.....	14
7.3 Ejemplos de datos.....	15
8. Referencias.....	16

1. Resumen ejecutivo

Ecommify requiere una arquitectura de datos capaz de soportar operaciones transaccionales confiables y consultas analíticas orientadas a la toma de decisiones. Para este diseño se selecciona la **Opción 1: Arquitectura Transaccional-Analítica**, en la cual PostgreSQL actúa como núcleo del módulo transaccional y MongoDB como complemento documental para datos flexibles y analíticos.

PostgreSQL se propone para pedidos, pagos, inventario, clientes, vendedores y productos maestros, debido a que estas entidades requieren consistencia fuerte, integridad referencial, restricciones declarativas y soporte transaccional ACID. El modelo relacional se normaliza como mínimo hasta tercera forma normal, con el propósito de reducir redundancias, evitar anomalías de actualización y preservar la calidad de los datos. A su vez, se incorporan capacidades avanzadas de PostgreSQL como JSONB, arrays, rangos temporales, PostGIS, pg_trgm, particionamiento por fecha y vistas materializadas para responder a cargas híbridas OLTP/OLAP.

MongoDB se plantea para catálogo enriquecido, reseñas extendidas, eventos de comportamiento de usuario y snapshots analíticos, donde la flexibilidad de esquema y la consulta documental aportan mayor valor. La solución integra decisiones de modelado conceptual y lógico, criterios PostgreSQL vs MongoDB, restricciones de negocio y estrategias de mitigación técnica para una implementación progresiva en Supabase, MongoDB Atlas y Google Colab.

1.1 Alcance del documento

Este documento consolida el diseño conceptual y lógico del módulo transaccional de Ecommify en PostgreSQL y el diseño preliminar del módulo NoSQL en MongoDB.

El alcance cubre requisitos, entidades, modelo ER, esquema relacional, tipos avanzados, extensiones, particionamiento, vistas materializadas, colecciones documentales, decisiones arquitectónicas y anexos técnicos preliminares.

1.2 Enfoque arquitectónico seleccionado

El enfoque seleccionado es:

Opción 1: Arquitectura Transaccional-Analítica. Este enfoque separa los datos críticos y transaccionales, administrados en PostgreSQL, de los datos flexibles o de consulta analítica complementaria, administrados en MongoDB. La decisión responde al equilibrio entre integridad referencial, consistencia ACID, flexibilidad documental y soporte para cargas OLTP/OLAP.

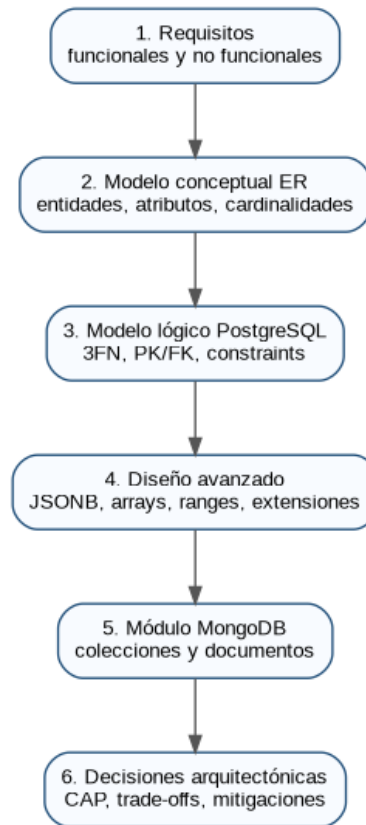


Figura 1. Flujo metodológico del diseño conceptual y lógico.

2. Análisis de requisitos

2.1 Contexto de Ecommify

Ecommify es una plataforma de comercio electrónico multivendedor especializada en productos tecnológicos. El sistema se apoya en el dataset Brazilian E-Commerce / Olist, compuesto por tablas de clientes, geolocalización, órdenes, ítems de órdenes, pagos, reseñas, productos, vendedores y categorías. El dominio exige registrar transacciones confiables y, simultáneamente, generar indicadores analíticos sobre ventas, clientes, categorías, vendedores, entregas y comportamiento de usuarios.

2.2 Requisitos funcionales

Código	Requisito funcional	Descripción
RF-01	Gestión de clientes	Registrar clientes, ubicación, ciudad, estado y datos asociados a compras.
RF-02	Gestión de vendedores	Registrar vendedores, ubicación y datos comerciales.
RF-03	Gestión de productos	Almacenar productos, categorías, dimensiones, peso, fotografías y especificaciones.

RF-04	Gestión de pedidos	Registrar órdenes de compra, fechas, estados y relación con clientes.
RF-05	Detalle de pedido	Registrar productos vendidos por pedido, vendedor, precio y costo de envío.
RF-06	Gestión de pagos	Registrar métodos de pago, cuotas, valores y relación con pedidos.
RF-07	Gestión de reseñas	Registrar calificaciones, comentarios y fechas de revisión.
RF-08	Gestión analítica	Consultar ventas por categoría, mes, cliente, vendedor y estado.
RF-09	Búsqueda de productos	Permitir búsqueda tolerante a errores tipográficos sobre nombres o categorías.
RF-10	Análisis geográfico	Analizar ubicación de clientes y vendedores para optimización logística.

2.3 Requisitos no funcionales

Código	Requisito no funcional	Descripción
RNF-01	Consistencia	Los pedidos, pagos e inventario deben mantener integridad transaccional.
RNF-02	Disponibilidad	El módulo transaccional en PostgreSQL deberá mantener una disponibilidad objetivo mínima del 99,5 % mensual durante el periodo de operación académica del prototipo, considerando las limitaciones de la capa gratuita de Supabase. Las operaciones críticas de consulta de pedidos, clientes y pagos deberán permanecer accesibles salvo ventanas de mantenimiento planificadas o restricciones propias del proveedor cloud.
RNF-03	Rendimiento OLTP	Las operaciones transaccionales frecuentes deberán cumplir tiempos de respuesta objetivo: consulta de pedido por order_id en ≤ 100 ms , consulta de pedidos por cliente en ≤ 300 ms , inserción de una orden con sus ítems en ≤ 500 ms y registro de pagos en ≤ 300 ms , bajo un volumen de referencia inicial de 100.000 pedidos y con índices definidos sobre claves primarias, foráneas y columnas de búsqueda frecuente.
RNF-04	Rendimiento OLAP	Las consultas analíticas usadas en dashboards deberán ejecutarse preferiblemente sobre vistas materializadas o tablas agregadas. Los reportes mensuales de ventas por categoría y segmentación de clientes deberán responder en ≤ 2 segundos sobre datos agregados y en ≤ 5 segundos cuando consulten particiones históricas, considerando el volumen académico del dataset Olist y las restricciones de Supabase Free Tier.
RNF-05	Escalabilidad	El diseño debe permitir crecimiento por volumen de órdenes y datos históricos.

RNF-06	Flexibilidad	Datos variables del catálogo y reseñas pueden almacenarse en formatos semiestructurados.
RNF-07	Auditabilidad	Las tablas principales deben incluir campos created_at y updated_at.
RNF-08	Mantenibilidad	El modelo debe estar documentado con diccionario de datos y scripts preliminares.

Nota metodológica

Las métricas propuestas son objetivos de referencia para el prototipo académico y deberán validarse mediante pruebas de carga controladas, análisis de planes de ejecución con EXPLAIN ANALYZE y monitoreo de consultas frecuentes.

2.4 Identificación de entidades del dominio

Entidad	Descripción	Módulo propuesto
Customer	Cliente que realiza compras en la plataforma.	PostgreSQL
Seller	Vendedor que ofrece productos.	PostgreSQL
Product	Producto publicado en el marketplace.	PostgreSQL + MongoDB
Product Category	Categoría comercial del producto.	PostgreSQL
Order	Pedido realizado por un cliente.	PostgreSQL
Order Item	Detalle de productos incluidos en un pedido.	PostgreSQL
Payment	Información de pagos asociados a una orden.	PostgreSQL
Review	Reseña y calificación de un pedido/producto.	MongoDB / PostgreSQL preliminar
Geolocation	Información geográfica de clientes y vendedores.	PostgreSQL + PostGIS
Inventory	Existencias disponibles por producto y vendedor.	PostgreSQL
Analytical View	Vista agregada para reportes.	PostgreSQL

2.5 Tabla de volúmenes de datos esperados

Los volúmenes son estimaciones iniciales que deben ajustarse con los resultados definitivos del EDA en Google Colab.

Entidad / tabla	Volumen inicial	Crecimiento	Carga	Observación
customers	Alto	Medio	OLTP	Consultas por cliente y ubicación.
sellers	Medio	Bajo	OLTP	Catálogo de vendedores.
products	Medio	Medio	OLTP / Catálogo	Puede tener atributos variables.
orders	Muy alto	Alto	OLTP / OLAP	Tabla crítica, candidata a particionamiento.
order_items	Muy alto	Alto	OLTP / OLAP	Base de análisis de ingresos por producto y vendedor.
payments	Alto	Alto	OLTP	Requiere consistencia con pedidos.
reviews	Alto	Alto	Consulta flexible	Candidata parcial a MongoDB.

inventory	Alto	Alto	OLTP	Requiere actualización frecuente.
materialized views	Medio	Medio	OLAP	Se refrescan periódicamente.

3. Diseño conceptual

El diseño conceptual identifica entidades, atributos, relaciones y cardinalidades desde la perspectiva del negocio, antes de entrar al detalle propio del DBMS. El objetivo es representar el dominio de Ecommify de forma legible para el equipo técnico y los interesados del negocio.

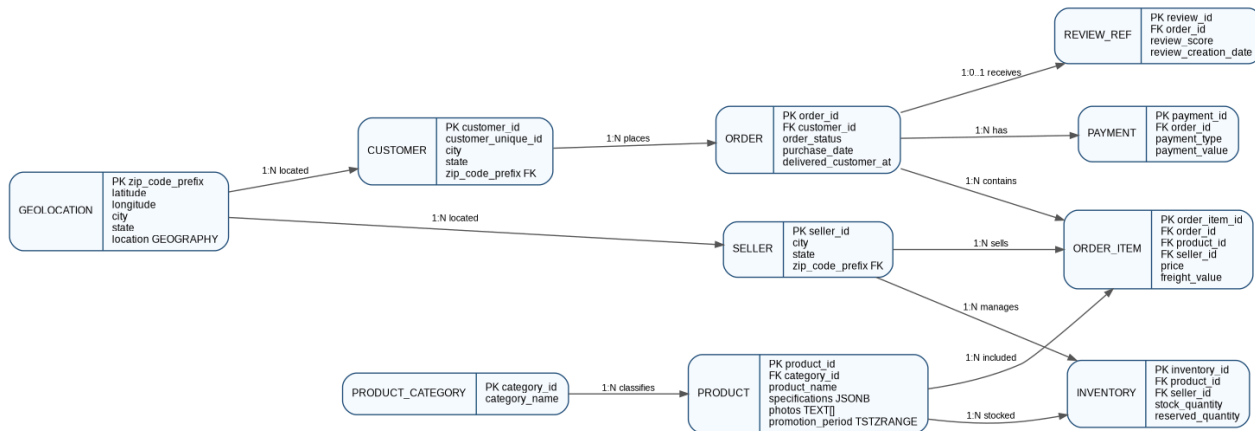


Figura 2. Diagrama ER conceptual propuesto para Ecommify.

3.1 Descripción de entidades y relaciones

Customer: Representa a los clientes que realizan compras. Se relaciona con Order porque un cliente puede realizar múltiples pedidos. Conserva ubicación para segmentación comercial y análisis geográfico.

Order: Entidad central del proceso de compra. Conecta clientes, pagos, reseñas e ítems de pedido. Por su volumen y crecimiento esperado, se propone particionamiento por fecha de compra.

Order Item: Detalle de productos incluidos en cada orden. Modela la relación entre pedidos, productos y vendedores, y sirve como base para reportes de ventas e ingresos.

Payment: Registra pagos asociados a órdenes. Debe mantenerse en PostgreSQL por requisitos de consistencia, trazabilidad y validación financiera.

Product: Representa productos del marketplace. Combina datos relacionales estables con tipos avanzados para atributos variables, fotografías y promociones.

Product Category: Clasificación comercial del producto. Permite agrupación de ventas y análisis por categoría.

Seller: Vendedor registrado en el marketplace. Se relaciona con order_items y con inventory para controlar ventas y existencias.

Inventory: Disponibilidad de productos por vendedor. Requiere consistencia fuerte para evitar cantidades negativas o ventas sin disponibilidad.

Review: Calificación y comentario asociado a una orden. Puede mantenerse como referencia en PostgreSQL y enriquecerse como documento en MongoDB.

Geolocation: Ubicación geográfica de clientes y vendedores. Puede aprovechar PostGIS para análisis de distancia y optimización logística.

3.2 Restricciones de negocio identificadas

Código	Restricción
RB-01	Un cliente puede tener múltiples pedidos, pero cada pedido pertenece a un único cliente.
RB-02	Un pedido debe tener al menos un producto asociado.
RB-03	Un producto puede ser vendido por varios vendedores.
RB-04	Un vendedor puede vender múltiples productos.
RB-05	Un pago debe estar asociado a una orden existente.
RB-06	El valor de un pago debe ser mayor que cero.
RB-07	La calificación de una reseña debe estar entre 1 y 5.
RB-08	El estado de una orden debe pertenecer a un conjunto controlado de valores.
RB-09	El inventario no puede tener cantidades negativas.
RB-10	Las fechas de entrega no pueden ser anteriores a la fecha de compra.
RB-11	Las órdenes históricas deben conservarse para análisis aunque no sean consultadas frecuentemente en OLTP.
RB-12	Los cambios sobre tablas críticas deben actualizar automáticamente el campo updated_at.

4. Diseño lógico - Módulo PostgreSQL

PostgreSQL se define como el núcleo transaccional de Ecommify. Se utiliza para entidades que requieren integridad referencial, consistencia fuerte, constraints, transacciones ACID y consultas relacionales. El modelo se normaliza como mínimo hasta tercera forma normal, manteniendo separadas entidades maestras y relaciones de asociación para evitar redundancias y anomalías de actualización.

4.1 Esquema relacional propuesto

Tabla: customers

Columna	Tipo	Restricción	Descripción
customer_id	UUID	PK	Identificador interno del cliente.
customer_unique_id	TEXT	NOT NULL	Identificador único del cliente.
zip_code_prefix	TEXT	FK	Código postal relacionado con geolocalización.
city	TEXT	NOT NULL	Ciudad del cliente.
state	CHAR(2)	NOT NULL	Estado o región.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización.

Tabla: sellers

Columna	Tipo	Restricción	Descripción
seller_id	UUID	PK	Identificador del vendedor.
zip_code_prefix	TEXT	FK	Código postal relacionado con geolocalización.
city	TEXT	NOT NULL	Ciudad del vendedor.
state	CHAR(2)	NOT NULL	Estado o región.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización.

Tabla: product_categories

Columna	Tipo	Restricción	Descripción
category_id	UUID	PK	Identificador de categoría.
category_name	TEXT	UNIQUE, NOT NULL	Nombre de categoría.
category_name_en	TEXT	NULL	Traducción o nombre homologado.

Tabla: products

Columna	Tipo	Restricción	Descripción
product_id	UUID	PK	Identificador del producto.
category_id	UUID	FK	Categoría del producto.
product_name	TEXT	NULL	Nombre del producto.
specifications	JSONB	NULL	Atributos variables del producto.
photos	TEXT[]	NULL	URLs o referencias de fotografías.
promotion_period	TSTZRANGE	NULL	Rango de vigencia promocional.
weight_g	NUMERIC(10,2)	CHECK >= 0	Peso en gramos.
length_cm	NUMERIC(10,2)	CHECK >= 0	Largo.
height_cm	NUMERIC(10,2)	CHECK >= 0	Alto.
width_cm	NUMERIC(10,2)	CHECK >= 0	Ancho.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización.

Tabla: orders

Columna	Tipo	Restricción	Descripción
order_id	UUID	PK	Identificador del pedido.
customer_id	UUID	FK, NOT NULL	Cliente que realiza el pedido.
order_status	TEXT	NOT NULL	Estado del pedido.
purchase_date	TIMESTAMPTZ	NOT NULL	Fecha de compra.
approved_at	TIMESTAMPTZ	NULL	Fecha de aprobación.
delivered_carrier_at	TIMESTAMPTZ	NULL	Fecha de entrega a transportadora.
delivered_customer_at	TIMESTAMPTZ	NULL	Fecha de entrega al cliente.
estimated_delivery_at	TIMESTAMPTZ	NULL	Fecha estimada de entrega.
created_at	TIMESTAMPTZ	DEFAULT now()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT now()	Fecha de actualización.

Tabla: order_items

Columna	Tipo	Restricción	Descripción
order_item_id	UUID	PK	Identificador del ítem.
order_id	UUID	FK, NOT NULL	Pedido asociado.
product_id	UUID	FK, NOT NULL	Producto vendido.
seller_id	UUID	FK, NOT NULL	Vendedor asociado.
shipping_limit_date	TIMESTAMPTZ	NULL	Fecha límite de envío.
price	NUMERIC(12,2)	CHECK > 0	Precio del producto.
freight_value	NUMERIC(12,2)	CHECK >= 0	Valor del envío.

Tabla: payments

Columna	Tipo	Restricción	Descripción
payment_id	UUID	PK	Identificador del pago.
order_id	UUID	FK, NOT NULL	Orden asociada.
payment_sequential	INT	NOT NULL	Secuencia del pago.
payment_type	TEXT	NOT NULL	Tipo de pago.
payment_installments	INT	CHECK >= 1	Número de cuotas.
payment_value	NUMERIC(12,2)	CHECK > 0	Valor pagado.

Tabla: inventory

Columna	Tipo	Restricción	Descripción
inventory_id	UUID	PK	Identificador del inventario.
product_id	UUID	FK, NOT NULL	Producto asociado.
seller_id	UUID	FK, NOT NULL	Vendedor asociado.
stock_quantity	INT	CHECK >= 0	Cantidad disponible.
reserved_quantity	INT	CHECK >= 0	Cantidad reservada.
updated_at	TIMESTAMPTZ	DEFAULT now()	Última actualización.

Tabla: geolocation

Columna	Tipo	Restricción	Descripción
zip_code_prefix	TEXT	PK	Prefijo de código postal.
latitude	NUMERIC(10,7)	NOT NULL	Latitud.
longitude	NUMERIC(10,7)	NOT NULL	Longitud.
city	TEXT	NOT NULL	Ciudad.
state	CHAR(2)	NOT NULL	Estado.
location	GEOGRAPHY(Point, 4326)	NULL	Punto geográfico para PostGIS.

4.2 Aplicación de tipos avanzados PostgreSQL

Tipo avanzado	Campo	Tabla	Justificación
JSONB	specifications	products	Permite almacenar atributos variables por categoría sin alterar constantemente el esquema.
TEXT[]	photos	products	Permite registrar múltiples fotografías por producto de forma compacta.
TSTZRANGE	promotion_period	products	Permite consultar productos con promoción vigente en un rango temporal.
GEOGRAPHY(Point, 4326)	location	geolocation	Permite cálculos de distancia entre clientes y vendedores.
ENUM o CHECK	order_status	orders	Controla estados válidos de una orden.

4.3 Extensiones PostgreSQL propuestas

Extensión	Uso en Ecommify	Justificación
PostGIS	Análisis geográfico de clientes y vendedores.	Permite calcular distancias y optimizar costos o tiempos de envío.
pg_trgm	Búsqueda de productos tolerantes a errores.	Mejora la experiencia de búsqueda cuando el usuario escribe con errores tipográficos.
pgcrypto	Generación de UUID y soporte criptográfico.	Facilita identificadores seguros y funciones criptográficas.
hstore	Atributos clave-valor simples.	Puede evaluarse para metadatos sencillos; JSONB se mantiene como opción principal.

4.4 Estrategia de particionamiento

La tabla orders se propone particionada por rango sobre purchase_date. La partición hot contendrá datos recientes de alta consulta operacional, mientras que las particiones cold conservarán datos históricos orientados a análisis. Este diseño mejora consultas por periodo, facilita mantenimiento y permite limpiar o archivar datos sin bloquear toda la tabla.

```
CREATE TABLE orders (
  order_id UUID NOT NULL,
  customer_id UUID NOT NULL,
  order_status TEXT NOT NULL,
  purchase_date TIMESTAMPTZ NOT NULL,
  approved_at TIMESTAMPTZ,
  delivered_carrier_at TIMESTAMPTZ,
  delivered_customer_at TIMESTAMPTZ,
  estimated_delivery_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (order_id, purchase_date)
) PARTITION BY RANGE (purchase_date);

CREATE TABLE orders_2024_01 PARTITION OF orders
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
```

5. Diseño lógico preliminar - Módulo MongoDB

MongoDB se propone como módulo complementario, no como reemplazo del núcleo transaccional. Su uso se orienta a datos flexibles, semiestructurados o de consulta documental, especialmente aquellos que no requieren integridad relacional estricta o que se benefician de documentos enriquecidos para lectura rápida.

Colección	Descripción	Justificación
product_catalog	Vista documental enriquecida del producto.	Facilita búsquedas flexibles y atributos variables.
product_reviews	Comentarios, reseñas extendidas y análisis textual.	Las reseñas pueden tener estructura flexible y crecer en volumen.
customer_events	Eventos de navegación, búsquedas y clics.	Datos de comportamiento no necesariamente transaccionales.
dashboard_snapshots	Resultados analíticos precalculados.	Útil para consultas rápidas desde frontends analíticos.

5.1 Ejemplo de documento: product_catalog

```
{
  "product_id": "uuid-product-001",
  "name": "Smartwatch deportivo",
  "category": {
    "category_id": "uuid-category-001",
    "name": "electronics"
  },
  "specifications": {
    "brand": "Generic",
    "color": "black",
    "battery_life": "7 days",
    "water_resistant": true
  },
  "photos": [
    "https://storage.ecommify/products/001/photo1.jpg",
    "https://storage.ecommify/products/001/photo2.jpg"
  ],
  "seller_summary": [
    {

```

```
    "seller_id": "uuid-seller-001",
    "city": "Sao Paulo",
    "state": "SP",
    "average_delivery_days": 4.2
  },
  "rating_summary": {
    "average_score": 4.6,
    "total_reviews": 128
  },
  "search_keywords": ["smartwatch", "reloj deportivo", "electronics"],
  "updated_at": "2026-05-24T00:00:00Z"
}
```

5.2 Ejemplo de documento: product_reviews

```
{
  "review_id": "uuid-review-001",
  "order_id": "uuid-order-001",
  "product_id": "uuid-product-001",
  "customer_id": "uuid-customer-001",
  "score": 5,
  "title": "Excelente producto",
  "message": "Llegó rápido y funciona correctamente.",
  "sentiment": "positive",
  "tags": ["delivery", "quality", "price"],
  "created_at": "2026-05-24T00:00:00Z"
}
```

5.3 Patrones de modelado MongoDB propuestos

Patrón	Aplicación
Embedding	Incluir datos de categoría, resumen de vendedores y resumen de reseñas dentro del catálogo.
Referencing	Mantener product_id, order_id y customer_id como referencias hacia PostgreSQL.
Schema flexibility	Permitir especificaciones diferentes según tipo de producto.
Precomputed views	Guardar snapshots analíticos para dashboards de consulta rápida.

6. Decisiones arquitectónicas justificadas

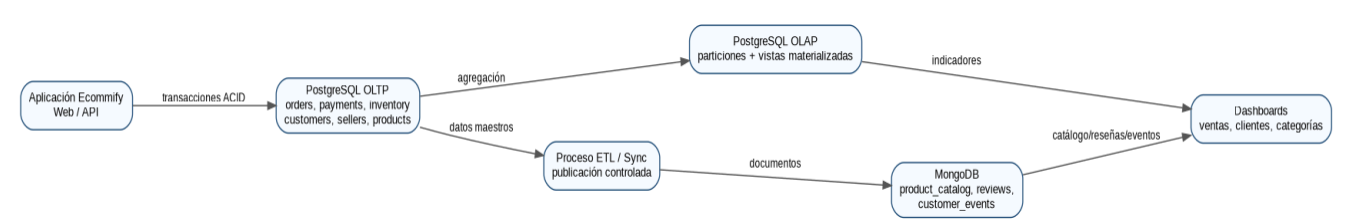


Figura 3. Arquitectura híbrida transaccional-analítica propuesta.

6.1 Matriz de decisión por entidad

Entidad	PostgreSQL	MongoDB	Criterio principal	Decisión
customers	Alto	Bajo	Consistencia e integridad	PostgreSQL
sellers	Alto	Bajo	Relación con órdenes e inventario	PostgreSQL
products	Alto	Medio	Maestro relacional + atributos flexibles	PostgreSQL + MongoDB
orders	Muy alto	Bajo	Transaccionalidad ACID	PostgreSQL
order_items	Muy alto	Bajo	Integridad con pedidos, productos y vendedores	PostgreSQL
payments	Muy alto	No	Consistencia financiera	PostgreSQL
inventory	Muy alto	Bajo	Control de stock	PostgreSQL
reviews	Medio	Alto	Texto flexible y análisis documental	MongoDB complementario
customer_events	Bajo	Alto	Alto volumen, estructura variable	MongoDB
analytics	Medio	Medio	Agregación y consulta rápida	PostgreSQL + MongoDB opcional

6.2 Análisis CAP aplicado

En la arquitectura propuesta, PostgreSQL se utiliza como sistema principal para datos transaccionales, priorizando consistencia y tolerancia razonable a fallos dentro del contexto de una base relacional administrada. Para operaciones críticas como pedidos, pagos e inventario, la consistencia es más importante que la disponibilidad eventual, ya que errores en estos datos pueden afectar directamente la operación del negocio.

MongoDB se propone para datos donde la disponibilidad, flexibilidad y escalabilidad horizontal pueden ser más relevantes que la consistencia inmediata. Por ejemplo, eventos de navegación, reseñas extendidas o snapshots de dashboards pueden aceptar modelos de consistencia eventual, siempre que el dato maestro permanezca controlado en PostgreSQL.

6.3 Trade-offs y estrategias de mitigación

Decisión	Beneficio	Trade-off	Mitigación
PostgreSQL como núcleo transaccional	Alta consistencia e integridad	Puede requerir optimización para grandes volúmenes	Índices, particionamiento y monitoreo.
Uso de JSONB en productos	Flexibilidad sin perder PostgreSQL	Puede dificultar validaciones estrictas	Validaciones de aplicación y constraints parciales.
Particionamiento de orders	Mejora consultas históricas y mantenimiento	Mayor complejidad operativa	Automatizar creación mensual de particiones.
Vistas materializadas	Mejora rendimiento analítico	Datos no siempre en tiempo real	Refresh programado según necesidad del negocio.
MongoDB para reseñas/eventos	Flexibilidad y escalabilidad	Consistencia eventual	Mantener IDs maestros sincronizados con PostgreSQL.

6.4 Flujo general de sincronización

1. La aplicación registra clientes, pedidos, pagos, productos e inventario en PostgreSQL.
2. PostgreSQL mantiene la integridad transaccional del negocio.
3. Las vistas materializadas consolidan información para analítica.
4. Un proceso de sincronización publica datos enriquecidos hacia MongoDB.
5. MongoDB sirve consultas flexibles de catálogo, reseñas y snapshots no críticos.
6. Los dashboards consumen datos agregados desde PostgreSQL y, opcionalmente, desde MongoDB.

7. Anexos

7.1 Diccionario de datos resumido

Campo	Tipo	Obligatorio	Clave	Descripción	Regla de negocio
orders.order_id	UUID	Sí	PK	Identificador del pedido	Debe ser único dentro de su partición.
orders.customer_id	UUID	Sí	FK	Cliente asociado	Debe existir en customers.
orders.purchase_date	TIMESTAMPTZ	Sí	Parte PK	Fecha de compra	No debería ser futura.
order_items.price	NUMERIC(12,2)	Sí	No	Precio del producto	Debe ser mayor que cero.
payments.payment_value	NUMERIC(12,2)	Sí	No	Valor pagado	Debe ser mayor que cero.
reviews.review_score	INT	Sí	No	Calificación	Debe estar entre 1 y 5.
inventory.stock_quantity	INT	Sí	No	Existencia disponible	No puede ser negativa.
products.specifications	JSONB	No	No	Atributos variables	Debe usar estructura documentada por categoría.
products.photos	TEXT[]	No	No	Fotografías del producto	Lista de URLs o rutas internas.
geolocation.location	GEOGRAPHY	No	No	Punto geográfico	Debe derivarse de latitud y longitud.

7.2 Scripts SQL preliminares

Los scripts presentados en este anexo no corresponden a la implementación final completa, hacen eco de una versión preliminar organizada por responsabilidades.

Sugerido	Propósito
00_extensions.sql	Crear extensiones: pgcrypto, postgis, pg_trgm, opcionalmente hstore.
01_schemas.sql	Crear esquemas lógicos, por ejemplo core, analytics, staging.
02_types_domains.sql	Crear tipos controlados, por ejemplo estados de orden, tipos de pago o dominios de validación.
03_tables_core.sql	Crear tablas maestras: clientes, vendedores, categorías, productos, geolocalización.
04_tables_transactions.sql	Crear tablas transaccionales: órdenes, ítems de orden, pagos, inventario, reseñas mínimas.

05_constraints.sql	Definir claves foráneas, CHECK, UNIQUE, NOT NULL adicionales si no se dejaron embebidos en las tablas.
06_indexes.sql	Crear índices para PK/FK, búsquedas frecuentes, GIN sobre JSONB, trigramas y geolocalización.
07_partitioning_orders.sql	Crear tabla orders particionada por fecha y particiones iniciales.
08_triggers_updated_at.sql	Crear función y triggers para actualizar updated_at.
09_materialized_views.sql	Crear vistas materializadas analíticas como mv_sales_by_category_monthly y mv_customer_segments.
10_seed_data.sql	Insertar datos de ejemplo mínimos para demostrar funcionamiento.
11_validation_queries.sql	Consultas de validación: conteos, joins principales, revisión de relaciones, pruebas básicas.
12_monitoring_queries.sql	Consultas preliminares para monitorear rendimiento OLTP/OLAP, queries lentas o uso de índices.

Con la intención de completar los ejemplos de datos y tener una aproximación acertada se agregan las siguientes tablas con sus correspondientes índices.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE TABLE product_categories (
    category_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category_name TEXT UNIQUE NOT NULL,
    category_name_en TEXT
);

CREATE TABLE products (
    product_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category_id UUID REFERENCES product_categories(category_id),
    product_name TEXT,
    specifications JSONB,
    photos TEXT[],
    promotion_period TSTZRANGE,
    weight_g NUMERIC(10,2) CHECK (weight_g >= 0),
    length_cm NUMERIC(10,2) CHECK (length_cm >= 0),
    height_cm NUMERIC(10,2) CHECK (height_cm >= 0),
    width_cm NUMERIC(10,2) CHECK (width_cm >= 0),
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_products_specifications_gin ON products USING GIN (specifications);
CREATE INDEX idx_products_name_trgm ON products USING GIN (product_name gin_trgm_ops);
```

7.3 Ejemplos de datos

```
INSERT INTO product_categories (category_name, category_name_en)
VALUES ('informatica_acessorios', 'computers_accessories');

INSERT INTO products (category_id, product_name, specifications, photos)
SELECT category_id,
    'Mouse inalámbrico',
    '{"brand":"Generic","connectivity":"wireless","color":"black"}'::jsonb,
    ARRAY['mouse_001_1.jpg', 'mouse_001_2.jpg']
FROM product_categories
WHERE category_name = 'informatica_acessorios';
```


8. Referencias

Guía de actividades. Unidad de PostgreSQL: Diseño relacional avanzado. Etapa 2: Entregable 1. Diseño conceptual y lógico de la base de datos del proyecto.

Teorey, T., Lightstone, S., Nadeau, T., & Jagadish, H. V. (2011). Database Modeling and Design: Logical Design (5th ed.). Morgan Kaufmann.

Juba, S., & Volkov, A. (2017). Learning PostgreSQL 10 (2nd ed.). Packt Publishing.

Riggs, S., Ciolli, G., & Meesala, S. K. (2019). PostgreSQL 11 Administration Cookbook. Packt Publishing.

Watt, A., & Eng, N. (2014). Database Design (2nd ed.). BCcampus Open Education.

PostgreSQL Global Development Group. PostgreSQL Documentation: Data Types, Extensions and DDL.

MongoDB, Inc. MongoDB Manual: Data Modeling Introduction.